Von der Idee zum Code

Algorithmisch denken und programmieren lernen

Die goldene Regel

Die goldene Regel

Niemals sofort Code schreiben!

1 — Die „Was muss ich mir merken?”-Frage

1 — Die „Was muss ich mir merken?”-Frage

- Variablen sind Merkzettel

1 — Die „Was muss ich mir merken?”-Frage

- Variablen sind Merkzettel
- Was muss ich mir für den nächsten Schritt merken? → jede Antwort = eine Variable
- Die Werte der Variablen sind der *Zustand* des Programms

1 — Die „Was muss ich mir merken?”-Frage

1 — Die „Was muss ich mir merken?”-Frage

- Beispiel: Gegeben ist eine Zeichenkette aus `L` (Land) und `W` (Wasser). Zähle die zusammenhängenden Landgruppen (“`LLWLLLWWWL`” → 3)

1 — Die „Was muss ich mir merken?”-Frage

- Beispiel: Gegeben ist eine Zeichenkette aus `L` (Land) und `W` (Wasser). Zähle die zusammenhängenden Landgruppen (“`LLWLLLWWWL`” → 3)
  - Durch die Zeichenkette iterieren

1 — Die „Was muss ich mir merken?”-Frage

- Beispiel: Gegeben ist eine Zeichenkette aus `L` (Land) und `W` (Wasser). Zähle die zusammenhängenden Landgruppen (“`LLWLLLWWWL`” → 3)
  - Durch die Zeichenkette iterieren
  - Zustandsvariable (Boolean), die sich merkt, ob man gerade auf einer „Insel” ist

1 — Die „Was muss ich mir merken?”-Frage

- Beispiel: Gegeben ist eine Zeichenkette aus `L` (Land) und `W` (Wasser). Zähle die zusammenhängenden Landgruppen (“`LLWLLLWWWL`” → 3)
  - Durch die Zeichenkette iterieren
  - Zustandsvariable (Boolean), die sich merkt, ob man gerade auf einer „Insel” ist
  - Bei jedem Wechsel auf `True` wird der Zähler um 1 erhöht

1 — Die „Was muss ich mir merken?”-Frage

1 — Die „Was muss ich mir merken?”-Frage

- Aufgabe: Prüfe, ob die Vorzeichen in einer Liste von Zahlen sich streng abwechseln

2 — Vom Ergebnis her denken

2 — Vom Ergebnis her denken

- Was will ich am Ende haben, und was brauche ich dafür?

2 — Vom Ergebnis her denken

- Was will ich am Ende haben, und was brauche ich dafür?
- … und was brauche ich *dafür*?

2 — Vom Ergebnis her denken

- Was will ich am Ende haben, und was brauche ich dafür?
- … und was brauche ich *dafür*?
- … und was brauche ich *dafür*?

2 — Vom Ergebnis her denken

- Was will ich am Ende haben, und was brauche ich dafür?
- … und was brauche ich *dafür*?
- … und was brauche ich *dafür*?
- Die Umkehrung dieser Auflistung ist der Algorithmus!

2 — Vom Ergebnis her denken

2 — Vom Ergebnis her denken

- Beispiel: Berechne den Durchschnitt einer Zahlenliste

2 — Vom Ergebnis her denken

- Beispiel: Berechne den Durchschnitt einer Zahlenliste
  - Für den Durchschnitt brauche ich Summe und Anzahl

2 — Vom Ergebnis her denken

- Beispiel: Berechne den Durchschnitt einer Zahlenliste
  - Für den Durchschnitt brauche ich Summe und Anzahl
  - Für die Summe muss ich alle Zahlen addieren

2 — Vom Ergebnis her denken

- Beispiel: Berechne den Durchschnitt einer Zahlenliste
  - Für den Durchschnitt brauche ich Summe und Anzahl
  - Für die Summe muss ich alle Zahlen addieren
  - Um alle Zahlen zu addieren, muss ich die Liste mit einer Schleife durchlaufen

2 — Vom Ergebnis her denken

- Beispiel: Berechne den Durchschnitt einer Zahlenliste
  - Für den Durchschnitt brauche ich Summe und Anzahl
  - Für die Summe muss ich alle Zahlen addieren
  - Um alle Zahlen zu addieren, muss ich die Liste mit einer Schleife durchlaufen
  - Die Anzahl der Schleifendurchläufe muss sich nach der Länge der Liste richten

2 — Vom Ergebnis her denken

- Beispiel: Berechne den Durchschnitt einer Zahlenliste
  - Für den Durchschnitt brauche ich Summe und Anzahl
  - Für die Summe muss ich alle Zahlen addieren
  - Um alle Zahlen zu addieren, muss ich die Liste mit einer Schleife durchlaufen
  - Die Anzahl der Schleifendurchläufe muss sich nach der Länge der Liste richten
  - …

2 — Vom Ergebnis her denken

2 — Vom Ergebnis her denken

- Aufgabe: Finde den Buchstaben, der in einem Wort am häufigsten vorkommt

3 — Ein konkretes Beispiel durchspielen

3 — Ein konkretes Beispiel durchspielen

- Gesetzmäßigkeiten des Problems am konkreten Beispiel ablesen und von dort verallgemeinern

3 — Ein konkretes Beispiel durchspielen

- Gesetzmäßigkeiten des Problems am konkreten Beispiel ablesen und von dort verallgemeinern
- Dabei sich selbst beim Denken beobachten!

3 — Ein konkretes Beispiel durchspielen

3 — Ein konkretes Beispiel durchspielen

- Beispiel: Prüfe, ob die Liste `[3, 5, 8, 2]` aufsteigend sortiert ist

3 — Ein konkretes Beispiel durchspielen

- Beispiel: Prüfe, ob die Liste `[3, 5, 8, 2]` aufsteigend sortiert ist
  - Vergleiche 3 und 5 → 5 ist größer ✅

3 — Ein konkretes Beispiel durchspielen

- Beispiel: Prüfe, ob die Liste `[3, 5, 8, 2]` aufsteigend sortiert ist
  - Vergleiche 3 und 5 → 5 ist größer ✅
  - Vergleiche 5 und 8 → 8 ist größer ✅

3 — Ein konkretes Beispiel durchspielen

- Beispiel: Prüfe, ob die Liste `[3, 5, 8, 2]` aufsteigend sortiert ist
  - Vergleiche 3 und 5 → 5 ist größer ✅
  - Vergleiche 5 und 8 → 8 ist größer ✅
  - Vergleiche 8 und 2 → 2 ist kleiner ❌

3 — Ein konkretes Beispiel durchspielen

- Beispiel: Prüfe, ob die Liste `[3, 5, 8, 2]` aufsteigend sortiert ist
  - Vergleiche 3 und 5 → 5 ist größer ✅
  - Vergleiche 5 und 8 → 8 ist größer ✅
  - Vergleiche 8 und 2 → 2 ist kleiner ❌
  - Beobachtung „Was habe ich getan?”:

3 — Ein konkretes Beispiel durchspielen

- Beispiel: Prüfe, ob die Liste `[3, 5, 8, 2]` aufsteigend sortiert ist
  - Vergleiche 3 und 5 → 5 ist größer ✅
  - Vergleiche 5 und 8 → 8 ist größer ✅
  - Vergleiche 8 und 2 → 2 ist kleiner ❌
  - Beobachtung „Was habe ich getan?”:
    - Jeweils linkes und rechtes Element verglichen

3 — Ein konkretes Beispiel durchspielen

- Beispiel: Prüfe, ob die Liste `[3, 5, 8, 2]` aufsteigend sortiert ist
  - Vergleiche 3 und 5 → 5 ist größer ✅
  - Vergleiche 5 und 8 → 8 ist größer ✅
  - Vergleiche 8 und 2 → 2 ist kleiner ❌
  - Beobachtung „Was habe ich getan?”:
    - Jeweils linkes und rechtes Element verglichen
    - Rechts größer als (oder gleich) links? Dann ✅, sonst ❌

3 — Ein konkretes Beispiel durchspielen

- Beispiel: Prüfe, ob die Liste `[3, 5, 8, 2]` aufsteigend sortiert ist
  - Vergleiche 3 und 5 → 5 ist größer ✅
  - Vergleiche 5 und 8 → 8 ist größer ✅
  - Vergleiche 8 und 2 → 2 ist kleiner ❌
  - Beobachtung „Was habe ich getan?”:
    - Jeweils linkes und rechtes Element verglichen
    - Rechts größer als (oder gleich) links? Dann ✅, sonst ❌
    - Ganze Liste ohne Widerspruch? Dann ✅, sonst ❌

3 — Ein konkretes Beispiel durchspielen

3 — Ein konkretes Beispiel durchspielen

- Aufgabe: Stelle Wechselgeld aus möglichst wenigen Münzen zusammen
  (z. B. 73 Cent mit gegebenen Münzen 1€ und 2 € sowie 50, 20, 10, 5, 2 und 1 Cent)

4 — Das Akkumulatormuster

4 — Das Akkumulatormuster

- Starte mit leerem/neutralem Ergebnis und baue es (meistens durch eine Schleife) Schritt für Schritt auf

4 — Das Akkumulatormuster

- Starte mit leerem/neutralem Ergebnis und baue es (meistens durch eine Schleife) Schritt für Schritt auf
- Typisches Muster:
  - Initialisierung der Ergebnisvariable
  - Schrittweiser Aufbau des Ergebnis in einer Schleife
  - Zurückgeben des akkumulierten Werts in der Ergebnisvariable

4 — Das Akkumulatormuster

4 — Das Akkumulatormuster

- Beispiel: Bestimme die Summe der Elemente von `[5, 3, 7]`

4 — Das Akkumulatormuster

- Beispiel: Bestimme die Summe der Elemente von `[5, 3, 7]`
  - Initialisiere eine Ergebnisvariable mit 0

4 — Das Akkumulatormuster

- Beispiel: Bestimme die Summe der Elemente von `[5, 3, 7]`
  - Initialisiere eine Ergebnisvariable mit 0
  - Schleife sieht 5 → bisheriges Ergebnis 0 + 5 = 5

4 — Das Akkumulatormuster

- Beispiel: Bestimme die Summe der Elemente von `[5, 3, 7]`
  - Initialisiere eine Ergebnisvariable mit 0
  - Schleife sieht 5 → bisheriges Ergebnis 0 + 5 = 5
  - Schleife sieht 3 → bisheriges Ergebnis 5 + 3 = 8

4 — Das Akkumulatormuster

- Beispiel: Bestimme die Summe der Elemente von `[5, 3, 7]`
  - Initialisiere eine Ergebnisvariable mit 0
  - Schleife sieht 5 → bisheriges Ergebnis 0 + 5 = 5
  - Schleife sieht 3 → bisheriges Ergebnis 5 + 3 = 8
  - Schleife sieht 7 → bisheriges Ergebnis 8 + 7 = 15

4 — Das Akkumulatormuster

- Beispiel: Bestimme die Summe der Elemente von `[5, 3, 7]`
  - Initialisiere eine Ergebnisvariable mit 0
  - Schleife sieht 5 → bisheriges Ergebnis 0 + 5 = 5
  - Schleife sieht 3 → bisheriges Ergebnis 5 + 3 = 8
  - Schleife sieht 7 → bisheriges Ergebnis 8 + 7 = 15
  - Gib das Ergebnis 15 zurück

4 — Das Akkumulatormuster

4 — Das Akkumulatormuster

- Aufgabe 1: Wörter nach Anfangsbuchstaben in einem Dictionary gruppieren

4 — Das Akkumulatormuster

- Aufgabe 1: Wörter nach Anfangsbuchstaben in einem Dictionary gruppieren
- Aufgabe 2: Run-Length-Encoding eines Strings (“`aabccc`” → “`a2b1c3`”)

5 — Grenzfälle zuerst behandeln

5 — Grenzfälle zuerst behandeln

- Eigentliche Logik des Codes wird oft einfacher, wenn man zunächst die Grenzfälle abräumt

5 — Grenzfälle zuerst behandeln

- Eigentliche Logik des Codes wird oft einfacher, wenn man zunächst die Grenzfälle abräumt
- Wenn Randbedingungen am Anfang geprüft wurden, kann der darauf-folgende Code Annahmen treffen

5 — Grenzfälle zuerst behandeln

5 — Grenzfälle zuerst behandeln

- Beispiel: Überlappen sich zwei Zeiträume?

5 — Grenzfälle zuerst behandeln

- Beispiel: Überlappen sich zwei Zeiträume?
  - Grenzfall 1: Endet der erste, bevor der zweite beginnt?

    → Keine Überlappung ❌

5 — Grenzfälle zuerst behandeln

- Beispiel: Überlappen sich zwei Zeiträume?
  - Grenzfall 1: Endet der erste, bevor der zweite beginnt?

    → Keine Überlappung ❌
  - Grenzfall 2: Endet der zweite, bevor der erste beginnt?

    → Keine Überlappung ❌

5 — Grenzfälle zuerst behandeln

- Beispiel: Überlappen sich zwei Zeiträume?
  - Grenzfall 1: Endet der erste, bevor der zweite beginnt?

    → Keine Überlappung ❌
  - Grenzfall 2: Endet der zweite, bevor der erste beginnt?

    → Keine Überlappung ❌
  - Alle anderen Fälle → Überlappung ✅

5 — Grenzfälle zuerst behandeln

5 — Grenzfälle zuerst behandeln

- Beispiel: Finde die größte von drei Zahlen `a`, `b`, `c`

5 — Grenzfälle zuerst behandeln

- Beispiel: Finde die größte von drei Zahlen `a`, `b`, `c`
  - Ist `a` größer als `b` und `c`? → dann (und nur dann) ist `a` die größte

5 — Grenzfälle zuerst behandeln

- Beispiel: Finde die größte von drei Zahlen `a`, `b`, `c`
  - Ist `a` größer als `b` und `c`? → dann (und nur dann) ist `a` die größte
  - Ist `b` größer als `c`? → dann (und nur dann) ist `b` die größte (denn `a` kann es ja nicht sein)

5 — Grenzfälle zuerst behandeln

- Beispiel: Finde die größte von drei Zahlen **a**, **b**, **c**
  - Ist **a** größer als **b** und **c**? → dann (und nur dann) ist **a** die größte
  - Ist **b** größer als **c**? → dann (und nur dann) ist **b** die größte (denn **a** kann es ja nicht sein)
  - Keiner der beiden oberen Fälle? → **c** ist die größte (denn **a** und **b** können es ja nicht sein)

5 — Grenzfälle zuerst behandeln

- Beispiel: Finde die größte von drei Zahlen **a**, **b**, **c**
  - Ist **a** größer als **b** und **c**? → dann (und nur dann) ist **a** die größte
  - Ist **b** größer als **c**? → dann (und nur dann) ist **b** die größte (denn **a** kann es ja nicht sein)
  - Keiner der beiden oberen Fälle? → **c** ist die größte (denn **a** und **b** können es ja nicht sein)
- Prinzip: Jeder Fall schließt Bedingungen aus, die der darauffolgende Fall dann als nicht zutreffend voraussetzen kann

5 — Grenzfälle zuerst behandeln

- Aufgabe: Ist ein gegebenes Jahr ein Schaltjahr oder nicht?

5 — Grenzfälle zuerst behandeln

- Aufgabe: Ist ein gegebenes Jahr ein Schaltjahr oder nicht?
  (Regel: Schaltjahre sind Jahre, die durch 4 teilbar sind; außer, wenn sie auch durch 100 teilbar sind, dann nicht; außer, wenn sie auch durch 400 teilbar sind, dann doch)

6 — Funktionale Dekomposition

6 — Funktionale Dekomposition

- Problem in Teilschritte zerlegen

6 — Funktionale Dekomposition

- Problem in Teilschritte zerlegen
- Jeden Teilschritt in eigener Funktion behandeln

6 — Funktionale Dekomposition

- Problem in Teilschritte zerlegen
- Jeden Teilschritt in eigener Funktion behandeln
- Fördert Fokus auf aktuellen Schritt

6 — Funktionale Dekomposition

- Problem in Teilschritte zerlegen
- Jeden Teilschritt in eigener Funktion behandeln
- Fördert Fokus auf aktuellen Schritt
- Erzwingt klare Schnittstellen und Verantwortlichkeiten der einzelnen Funktionen

6 — Funktionale Dekomposition

- Problem in Teilschritte zerlegen
- Jeden Teilschritt in eigener Funktion behandeln
- Fördert Fokus auf aktuellen Schritt
- Erzwingt klare Schnittstellen und Verantwortlichkeiten der einzelnen Funktionen
- „Teile und herrsche” (”divide and conquer”)

6 — Funktionale Dekomposition

- Beispiel: Prüfe, ob das Passwort `GP$6?AU25` „sicher” ist

6 — Funktionale Dekomposition

- Beispiel: Prüfe, ob das Passwort `GP$6?AU25` „sicher” ist
  - Hat es mindestens acht Zeichen? → ✅
  - Enthält es einen Großbuchstaben? → ✅
  - Enthält es einen Kleinbuchstaben? → ❌
  - Enthält es eine Ziffer? → ✅

6 — Funktionale Dekomposition

- Beispiel: Prüfe, ob das Passwort `GP$6?AU25` „sicher” ist
  - Hat es mindestens acht Zeichen? → ✅
  - Enthält es einen Großbuchstaben? → ✅
  - Enthält es einen Kleinbuchstaben? → ❌
  - Enthält es eine Ziffer? → ✅
  - Hauptfunktion kombiniert alle vier Prüfungen:

    ✅ und ✅ und ❌ und ✅ = ❌

Checkliste vor jeder Aufgabe (1/2)

Checkliste vor jeder Aufgabe (1/2)

- **Was geht rein?** Welche Eingaben braucht das Programm (Zahl, Text, Liste, …)?

Checkliste vor jeder Aufgabe (1/2)

- **Was geht rein?** Welche Eingaben braucht das Programm (Zahl, Text, Liste, …)?
- **Was kommt raus?** Was soll das Ergebnis sein (Boolean, Anzahl, Dictionary, …)?

Checkliste vor jeder Aufgabe (1/2)

- **Was geht rein?** Welche Eingaben braucht das Programm (Zahl, Text, Liste, …)?
- **Was kommt raus?** Was soll das Ergebnis sein (Boolean, Anzahl, Dictionary, …)?
- **Konkretes Beispiel**: Nimm ein einfaches Beispiel und löse es von Hand auf Papier. Beobachte deine einzelnen Denkschritte!

Checkliste vor jeder Aufgabe (1/2)

- **Was geht rein?** Welche Eingaben braucht das Programm (Zahl, Text, Liste, …)?
- **Was kommt raus?** Was soll das Ergebnis sein (Boolean, Anzahl, Dictionary, …)?
- **Konkretes Beispiel**: Nimm ein einfaches Beispiel und löse es von Hand auf Papier. Beobachte deine einzelnen Denkschritte!
- **Grenzfälle**: Was passiert bei leerer Eingabe, bei ungültigem String, bei negativen Zahlen?

Checkliste vor jeder Aufgabe (2/2)

Checkliste vor jeder Aufgabe (2/2)

- **Was wiederholt sich?** Gibt es ein Muster? Brauche ich eine Schleife?

Checkliste vor jeder Aufgabe (2/2)

- **Was wiederholt sich?** Gibt es ein Muster? Brauche ich eine Schleife?
- **Was muss ich mir merken?** Welche Variablen brauche ich? Auf welcher Ebene brauche ich sie? Was sind geeignete Namen?

Checkliste vor jeder Aufgabe (2/2)

- **Was wiederholt sich?** Gibt es ein Muster? Brauche ich eine Schleife?
- **Was muss ich mir merken?** Welche Variablen brauche ich? Auf welcher Ebene brauche ich sie? Was sind geeignete Namen?
- **Erst deutsch*, dann Python***: Schreib die Schritte zunächst in natürlicher Sprache auf, bis sie so klein sind, dass sie sich direkt in eine oder wenige Zeile(n) Code übersetzen lassen.

*** Welche (Programmier-)Sprache, ist natürlich völlig egal!**

Sonstiges

Alle Materialien dieses Kurses:



Übersicht aller meiner Kurse:



„If in physics there is something you don’t understand, you can always hide behind the uncharted depths of nature. You can always blame god. You didn’t make it so complex yourself.
But if your program doesn’t work, there is no-one to hide behind. You cannot hide behind an obstinate nature. […]
If it doesn’t work then you’ve messed up.”

– Edsger Dijkstra